



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3133 – HIGH PERFORMANCE DATA PROCESSING SECTION 2

PROJECT 2

**SHOPEE APPLICATION REAL-TIME SENTIMENT ANALYSIS USING APACHE
SPARK AND KAFKA**

LECTURER: DR. SEAH CHOON SEAN

SUBMISSION DATE:

STUDENT NAME	MATRIC NO
CHERYL CHEONG KAH VOON	A23CS0060
CHAU YING JIA	A23CS0213
LAU YEE WEN	A23CS0099
POH LOK YEE	A23CS0262

Table of Contents

1.0 Introduction	4
1.1 Project Background	4
1.2 Objectives	4
1.3 Scope	5
2. Data Acquisition & Preprocessing.....	5
2.1 Data Source Description.....	5
2.2 Data Collection Method	6
2.3 Data Cleaning and NLP Preprocessing	7
2.4 Final Dataset Description	8
3.0 Sentiment Model Development.....	9
3.1 Model Choice	9
3.2 Training Process	9
3.2.1 Multinomial Naive Bayes Training	10
3.2.2 Logistic Regression Training	11
3.2.3 Linear Support Vector Machine (Linear SVM) Training.....	11
3.3 Evaluation Metrics	11
3.4 Model Performance Comparison	12
3.5 Best Model Selection	13
4.0 Apache System Architecture	13
4.1 Overall Apache System Architecture.....	14
4.2 Docker-Based Deployment Environment	15
4.3 Kafka Streaming Layer	15
4.4 Kafka Producer for Review Data Ingestion.....	15
4.5 Spark Structured Streaming Processing Layer.....	16
4.6 Elasticsearch Storage Layer	17
4.7 Summary of Apache System Architecture	18
5.0 Dashboard Analysis and Results	18
5.1 Batch Sentiment Overview	19
5.2 Detailed Batch Prediction Analysis	20
5.3 Streaming Sentiment Overview	21
5.3.1 Kibana Streaming Output Validation.....	21

5.3.2 Power BI Streaming Sentiment Overview	23
5.4 Batch and Streaming Comparison	25
5.5 Summary	26
6.0 Optimization and Comparison	26
6.1 Batch and Streaming Comparison	27
6.2 Model Optimization and Improvement	28
6.3 Architecture Optimization and Improvement	28
6.4 Overall Discussion	29
7.0 Conclusion & Future Work	29
7.1 Conclusion	29
7.2 Improvement	30
References	31
Appendix	31

1.0 Introduction

1.1 Project Background

The rapid growth of e-commerce has changed the way consumers shop and interact with online platforms. Mobile shopping applications have become one of the primary channels for purchasing products, making payments, tracking deliveries, and communicating with sellers. As one of the leading e-commerce platforms in Malaysia, Shopee Malaysia receives a large number of customer reviews through its Google Play application every day. These reviews contain valuable feedback regarding application performance, user experience, payment services, delivery processes, promotional campaigns, and overall customer satisfaction.

The continuous increase in user-generated reviews makes manual analysis impractical and time-consuming (Jain & Pamula, 2020). Businesses require automated solutions that can process large volumes of textual data efficiently while providing timely insights into customer opinions. Sentiment analysis like Natural Language Processing (NLP) enables customer reviews to be classified into positive, negative, and neutral sentiments. This allows organizations to better understand customer satisfaction and identify issues that require improvement.

Although sentiment analysis has been widely adopted, many existing studies rely on offline or batch processing, where customer reviews are analysed only after they have been collected. Such an approach is less suitable for applications that continuously generate new customer feedback and require timely responses. Recent studies have demonstrated that integrating Apache Kafka and Apache Spark Structured Streaming enables real-time processing of continuously generated textual data, providing more timely insights than traditional batch processing (Poornima et al., 2025). Motivated by this need, this project proposes a real-time sentiment analysis system to analyse customer reviews from the Shopee Malaysia mobile application.

1.2 Objectives

The objectives of this project are:

1. To collect customer reviews of the Shopee Malaysia mobile application from Google Play.
2. To preprocess and clean the collected review data using Natural Language Processing (NLP) techniques, including text cleaning, tokenization, stop-word removal, and lemmatization.
3. To develop and evaluate at least two sentiment classification models for classifying customer reviews into positive, negative, and neutral sentiments.

4. To implement a real-time sentiment analysis pipeline using Apache Kafka and Apache Spark Structured Streaming.
5. To store sentiment classification results in Elasticsearch and visualize customer sentiment through dashboards.
6. To compare the performance of batch processing and streaming processing within the sentiment analysis pipeline.

1.3 Scope

This project focuses on customer reviews collected from the Shopee Malaysia mobile application available on Google Play. Approximately 10,000 publicly available reviews are collected using the Google Play Scraper library. The reviews include customer feedback related to application performance, shopping experience, payment services, delivery processes, promotional campaigns, and software updates.

The collected reviews are preprocessed using Natural Language Processing (NLP) techniques, including text cleaning, tokenization, stop-word removal, and lemmatization to produce a standardized dataset suitable for sentiment analysis. The project develops and compares at least two sentiment classification models to classify reviews into positive, negative, and neutral sentiment categories.

To support real-time analytics, Apache Kafka is used for streaming review data while Apache Spark Structured Streaming performs real-time sentiment classification using the selected model. The prediction results are stored in Elasticsearch and visualized through dashboards. Finally, the project compares the performance of batch processing and streaming processing to evaluate the effectiveness of the proposed real-time sentiment analysis pipeline.

2. Data Acquisition & Preprocessing

This section describes the process of collecting and preparing Shopee Malaysia Google Play reviews for sentiment analysis. The objective of this stage is to acquire a large collection of customer reviews and transform the raw textual data into a clean and standardized dataset suitable for machine learning and real-time streaming. The preprocessing pipeline focuses on improving data quality and ensuring that the same cleaned dataset can be reused throughout the project for model training, batch prediction, and streaming analysis.

2.1 Data Source Description

The primary data source selected for this project was the Google Play Store review of Shopee Malaysia mobile application. Google Play allows users to submit ratings and written reviews after using an application which providing publicly available customer feedback that reflects their experiences with the application. These reviews cover various aspects of the Shopee

mobile application, including shopping experience, application performance, payment services, promotional campaigns, delivery services, and customer support. As one of the leading e-commerce platforms in Malaysia, Shopee continuously receives customer feedback from Malaysian users which making Google Play reviews a suitable source of real-world textual data for sentiment analysis.

2.2 Data Collection Method

The data collection process was implemented using Python together with the Google Play Scraper library. Before collecting the reviews, the application package name (com.shopee.my) was verified to ensure that the reviews were retrieved from the official Shopee Malaysia application. The scraper continuously requested review pages from the Google Play Store using pagination until the target of 10,000 customer reviews was successfully collected.

Only information relevant to the objectives of this project was retained during the collection process. These attributes include the review identifier, customer rating, review date, application version, helpful vote count, and original review text. Personal information such as usernames, profile images, and developer replies was excluded because it was not required for sentiment analysis.

After the data collection process was completed, the collected reviews were exported as `shopee_my_reviews_raw.csv`, which serves as the raw input for the preprocessing stage. A summary of the collected raw data is presented in Table 2.1 below.

Table 2.1: Summary of Collected Raw Data

Attribute	Description
Data Source	Google Play Store
Application	Shopee Malaysia
Collection Tool	Google Play Scraper
File Format	CSV
Number of Records	10,000 reviews
Number of Attributes	6 columns

The raw review data contains customer review information together with the metadata retrieved from the Google Play Store. The attributes collected are described in Table 2.2 below.

Table 2.2: Description of Raw Data Attributes

Attribute	Data Type	Description	Example
Review ID	String	Unique identifier assigned to each Google Play review	12c34620-7baa-4041-85a9-2fc1ff43a864

Rating	Integer	Customer rating of the application (1–5 stars)	5
Review Date	Date	Date when the review was submitted	2026-06-28
App Version	String	Version of the Shopee application used by the reviewer	3.56.15
Thumbs Up Count	Integer	Number of users who marked the review as helpful	5
Original Text	Text	Original customer review collected from Google Play	"Fast delivery and many vouchers."

2.3 Data Cleaning and NLP Preprocessing

The raw Google Play reviews contained duplicate records, inconsistent text formatting, URLs, punctuation, emojis, and other unnecessary characters that could negatively affect sentiment classification performance. Therefore, a rule-based preprocessing pipeline was developed using Python, Pandas, and NLTK to improve data quality before model training.

The preprocessing pipeline consisted of the following steps:

1. Duplicate review IDs and duplicate review texts were removed to eliminate redundant records.
2. Missing values, empty reviews, and invalid reviews containing only symbols or emojis were removed.
3. Reviews containing fewer than three words were excluded because they provide limited semantic information for sentiment analysis.
4. All review text was converted to lowercase to ensure consistent word representation.
5. URLs, email addresses, HTML tags, punctuation, numerical values, emojis, and special characters were removed.
- 6.
7. English and common Malay stop words were removed to eliminate frequently occurring words that contribute little semantic meaning.
8. English lemmatization was applied to reduce words to their root forms, improving vocabulary consistency across the dataset.

After preprocessing, a new column named `cleaned_text` was generated while preserving the original review text for future reference. The preprocessing pipeline reduced the number of records from 10000 raw reviews to 6395 cleaned reviews. The final cleaned dataset was exported as `data/cleaned_data.csv`. This cleaned dataset forms the primary input for the

sentiment model development phase and is also used as the data source for Apache Kafka during streaming analysis.

2.4 Final Dataset Description

The final dataset consists of 6395 row records of cleaned Shopee Malaysia Google Play reviews prepared for both batch and streaming sentiment analysis. Each record contains the original review together with its corresponding preprocessed text and associated metadata required for subsequent analysis.

The final dataset contains the attributes that show in Table 2.3 below.

Table 2.3: Cleaned Dataset Descriptions

Attribute	Description
Review ID	Unique identifier of each Google Play review
Rating	Customer rating (1–5 stars)
Review Date	Date the review was submitted
App Version	Shopee application version
Thumbs Up Count	Number of users marking the review as helpful
Original Text	Original customer review
Cleaned Text	Review after NLP preprocessing

This dataset provides a consistent input to generate sentiment labels and train classification models, while also serving as the streaming data source for Apache Kafka and Apache Spark Structured Streaming. This ensures that both batch processing and streaming processing operate on the same standardized dataset, enabling a fair comparison of system performance.

3.0 Sentiment Model Development

This section describes the development of the sentiment classification models. Three models which are Multinomial Naive Bayes, Logistic Regression and Linear Support Vector Machine (SVM) were used in this project.

3.1 Model Choice

In this project, three widely used machine learning algorithms were selected for the comparison which are **Multinomial Naive Bayes (MNB)**, **Logistic Regression (LR)** and **Linear Support Vector Machine (Linear SVM)**. These models are often used in emotional analytics because they are effective in processing high-dimensional text data while maintaining relatively low computational complexity.

Multinomial Naive Bayes were chosen because it is specially designed for probabilistic text classification and runs efficiently on the sparse feature representation generated by TF-IDF. Its simple probability framework can achieve fast model training and produce competitive classification performance, making it a popular baseline model for emotional analysis.

Logistic Regression is chosen because it is a powerful linear classifier and shows strong performance in text classification tasks. It estimates the probability of each emotional category through linear decision-making boundaries, which usually provides good generalization ability for invisible data.

The Linear SVM is selected because it constructs an optimal separation hyperplane by maximizing the margins between different mood categories. This feature makes Linear SVM particularly suitable for high-dimensional text data sets and leads to its wide use in natural language processing applications.

Although these three algorithms belong to the traditional machine learning category, they represent different learning mechanisms. Multinomial Naive Bayes adopts the probability method, Logistic regression adopts the linear classification model, and Linear SVM adopts the maximum margin principle. Comparing these models can comprehensively evaluate different machine learning methods while ensuring a fair comparison under the same experimental conditions.

3.2 Training Process

Before training the machine learning models, the dataset is prepared for sentiment classification. The initial rating was converted into three emotional categories which are negative, neutral and positive. Ratings 1 to 2 are marked as negative, ratings 3 are marked as neutral, and ratings 4 to 5 are marked as positive. Each emotion class uses LabelEncoder in the Scikit-learn library to convert digital labels. Coding labels allow machine learning algorithms to process classified emotional targets in a digital format, in which each emotion class is represented by an integer value.

The dataset shows that the distribution of emotions is unbalanced. Positive comments represent the majority of classes, with 3,945 samples (61.69%), followed by negative comments, with 2,203 samples (34.45%). The neutral class contains only 247 samples (3.86%), making it a minority in the dataset. This imbalance may affect the model's ability to correctly classify minority emotional categories, especially neutral comments. To solve the problem of class imbalance, the `class_weight='balanced'` parameter is used to apply class weights in Logistic regression and Linear SVM models. This makes the model pay more attention to minorities during the training. Multinomial Naive Bayes uses Laplace's smooth probability method for training.

The cleaned textual reviews stored in the `cleaned_text` feature were then transformed into numerical representations using Term Frequency-Inverse Document Frequency (TF-IDF) vectorization. The TF-IDF Vectorizer was configured with a maximum of 10,000 features and extracted both unigram and bigram features using `ngram_range=(1,2)`. The `sublinear_tf=True` parameter was applied to reduce the effect of frequently occurring words by using logarithmic term frequency scaling. In addition, `min_df=2` was used to remove terms that appeared only once in the entire dataset, helping to reduce noise in the feature space.

After feature extraction, the dataset was divided into training, validation, and testing sets using stratified splitting. The data was separated into 70% training data, 10% validation data, and 20% testing data while maintaining the original distribution of sentiment classes in each subset. The TF-IDF vectorizer was fitted only on the training data, and the same transformation was applied to the validation, and testing sets to prevent data leakage.

Three machine learning models were trained and evaluated under the same experimental conditions, including Multinomial Naive Bayes, Logistic Regression, and Linear Support Vector Machine (Linear SVM). The testing dataset was used for the final performance comparison between the models.

3.2.1 Multinomial Naive Bayes Training

The first pipeline used Multinomial Naive Bayes (MNB) as the baseline classification model. The algorithm applies Bayes' theorem to calculate the probability of each emotion class according to the extracted text characteristics.

During the training, the model learned the probability distribution of words in each emotional category from the TF-IDF characteristic matrix. To deal with words that may not appear in some emotional classes, Laplace smoothly applies $\alpha=0.1$. This can prevent the zero-probability problem when predicting invisible text samples.

The Multinomial Naive Bayes model was selected as a baseline because it is usually used for text classification tasks and provides fast training performance with low computing requirements.

3.2.2 Logistic Regression Training

The second pipeline implements the logical regression of emotional classification. Logical regression learns the relationship between TF-IDF characteristics and emotional classes by estimating the probability of each class according to the input characteristics.

In order to reduce over-fitting, L2 regularization is applied using the parameter "C=0.5". The smaller C value increases the regularization effect, so that the model can avoid over-reliance on specific words in the training data.

Since the dataset contains an imbalance between emotional classes, `class_weight='balanced'` is applied. During training, this automatically assigns higher weights to lower-frequency classes and lower weights to more frequent classes. Therefore, the model is encouraged to improve the predictive performance of minority groups, especially neutral categories.

The maximum training iteration value of the logical regression model is 1000 to ensure that the optimization process reaches convergence.

3.2.3 Linear Support Vector Machine (Linear SVM) Training

The third pipeline applies the linear support vector machine (Linear SVM) for emotional classification. Linear SVM aims to find the decision-making boundaries that separate different emotional classes by maximizing the margins between classes.

The Linear SVM model was trained using a linear kernel approach with the regularization parameter $C=0.5$. The regularization parameter controls the trade-off between maximizing the classification margin and minimizing training errors. A balanced class weighting strategy (`class_weight='balanced'`) was incorporated to address the unequal distribution of sentiment classes by increasing the penalty contribution of misclassifying minority-class samples.

Linear SVM is suitable for this task, because TF-IDF generates high-dimensional and sparse text features. The model is trained using the generated TF-IDF matrix, which is later used to predict mood classes from the test data set.

After training all three pipelines, predictions were generated on the independent testing dataset. The performance of each model was subsequently compared using classification metrics including accuracy, precision, recall, and F1-score to determine the most effective approach for sentiment classification.

3.3 Evaluation Metrics

The performance of the well-trained emotional classification model was evaluated using a test dataset containing 1279 samples. Four classification indicators are used to compare the performance of Multinomial Naive Bayes, Logical Regression and Linear SVM, including accuracy, precision, recall and F1 score.

Accuracy measures the total proportion of samples correctly classified in the test dataset. Precision evaluates the correctness of positive predictions, and recall measures the ability of the model to correctly identify samples from each emotional category. F1-score combines accuracy and recall into a measurement to provide a balanced evaluation of model performance.

Since the dataset contains an unbalanced emotional distribution, the weighted F1 score is considered the main evaluation indicator. Unlike accuracy, the weighted F1 score considers the contribution of each emotional category according to the number of its samples, providing a more representative measure of the overall classification performance. Macro F1 scores are also considered to assess whether the model can be consistent in all emotional categories, including minorities.

3.4 Model Performance Comparison

Table 3.1 Performance Comparison of Sentiment Classification Models

Model	Accuracy	Precision	Recall	F1-Score	Macro Average F1-score
Multinomial Naive Bayes	88.19%	0.8538	0.8819	0.8663	0.59
Logistic Regression	84.75%	0.8629	0.8475	0.8475	0.63
Linear SVM	86.40%	0.8574	0.8640	0.8592	0.64

Table 3.2 Pre-class performance by for Multinomial Naive Bayes

Class	Precision	Recall	F1-Score	Support
Negative	0.79	0.91	0.85	441
Neutral	0.00	0.00	0.00	49
Positive	0.94	0.92	0.93	789

Table 3.3 Pre-class performance by for Logistic Regression

Class	Precision	Recall	F1-Score	Support
Negative	0.78	0.87	0.82	441
Neutral	0.15	0.18	0.16	49
Positive	0.96	0.88	0.92	789

Table 3.4 Pre-class performance by for Linear SVM

Class	Precision	Recall	F1-Score	Support
Negative	0.79	0.87	0.83	441
Neutral	0.26	0.14	0.18	49
Positive	0.93	0.90	0.92	789

Table 3.1 shows the overall performance comparison between the three emotional classification models. Multinomial Naive Bayes achieved the highest accuracy rate of 88.19% and the highest weighted F1 score of 0.8663, indicating the strong overall classification performance. However, the model shows limitations in predicting neutral emotions, with an F1 score of 0.00. This shows that the model is mainly influenced by the majority of emotional classes and cannot effectively identify samples of a minority class.

Compared with Multinomial Naive Bayes, the accuracy rate of Logistic regression is lower at 84.75%, but it provides better performance in minority groups. The application of class weighting helps the model to improve its ability to identify neutral comments.

Linear SVM achieved an accuracy rate of 86.40% and obtained the highest macro F1 score of 0.64 in the three models. This shows that Linear SVM provides the most balanced classification performance in different emotional categories.

Overall, the results show that each model has different advantages. Multinomial Naive Bayes provides the best overall prediction performance based on weighted indicators, while Linear SVM shows better balance in mood categories.

3.5 Best Model Selection

The final emotional classification model is selected according to the weighted F1 score and overall predictive performance. According to the comparison results, Multinomial Naive Bayes has the highest F1 weighted score of 0.8663 and the highest accuracy rate of 88.19%.

Multinomial Naive Bayes was selected as the final classification model for its strong overall performance and computational efficiency. The model is suitable for integration into the emotional analysis pipeline, because it provides fast training and prediction speed while maintaining high classification accuracy.

Although Linear SVM achieves a higher macro F1 score, it chooses Multinomial Naive Bayes because the main goal of the system is to provide reliable overall mood prediction for dashboard visualization and analysis. Future improvements can focus on dealing with class imbalances and improving minority predictions, especially for neutral sentiment.

4.0 Apache System Architecture

This section describes the Apache-based system architecture developed for the real-time sentiment analysis pipeline. The architecture integrates Apache Kafka, Apache Spark Structured Streaming, and Elasticsearch to enable real-time ingestion, processing, and storage of Shopee Malaysia customer review data. The cleaned dataset prepared in the previous stage is used as the input source for streaming, while the trained sentiment classification model is applied during the streaming process to generate prediction results. This architecture supports

continuous sentiment monitoring and provides the foundation for dashboard visualization and performance comparison between batch and streaming processing.

4.1 Overall Apache System Architecture

Figure 4.1 shows the overall Apache system architecture for the real-time sentiment analysis pipeline. The workflow begins with the cleaned Shopee review dataset, which was produced after data acquisition and NLP preprocessing. The cleaned dataset is streamed into Apache Kafka through a Python-based Kafka producer. Kafka then stores the incoming review messages in a topic named sentiment-topic, which acts as the communication channel between the producer and the processing layer.

Apache Spark Structured Streaming consumes the messages from the Kafka topic and processes the review data in micro-batches. During this stage, Spark parses the incoming JSON messages, extracts the required review attributes, and applies the trained sentiment classification model to the cleaned_text field. The prediction output is then stored in both streaming_prediction.csv and Elasticsearch under the sentiment_predictions index. This architecture allows the system to process customer review data continuously and supports real-time sentiment monitoring.

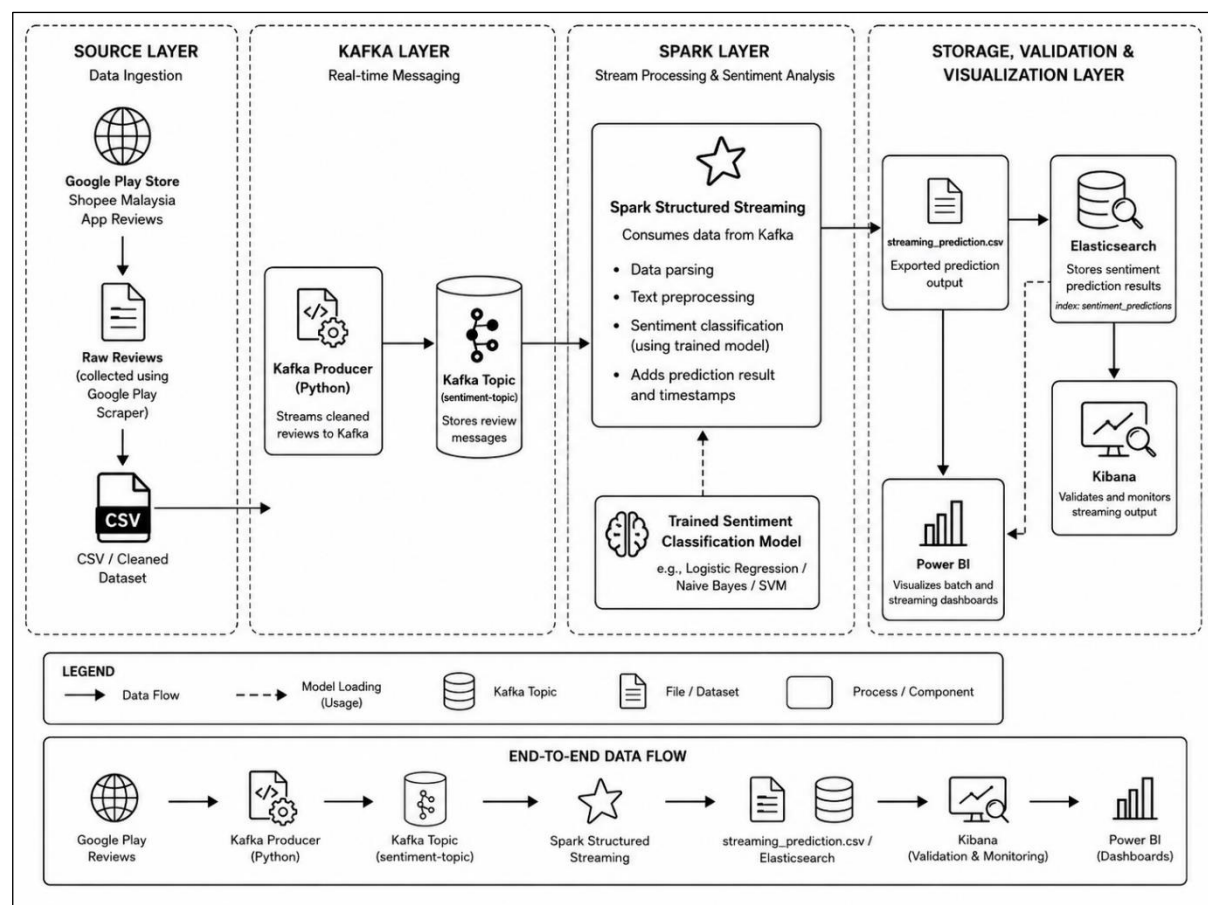


Figure 4.1 Overall Apache System Architecture for the Real-Time Sentiment Analysis Pipeline

4.2 Docker-Based Deployment Environment

Figure 4.2 shows the Docker containers used to run the system components required for the streaming pipeline. Docker was used to simplify the deployment of Zookeeper, Kafka, Elasticsearch, and Kibana in a local environment. Instead of installing and configuring each service separately, Docker Compose allows all required services to be started together using a single configuration file.

In this architecture, Zookeeper supports Kafka coordination, Kafka handles the streaming messages, Elasticsearch stores the sentiment prediction results, while Kibana provides an interface to validate and monitor the indexed streaming records. The stored and exported prediction outputs can then be used for further dashboard analysis in Power BI. As shown in Figure 4.2, all required containers were successfully started, which confirms that the environment was ready for real-time data ingestion, processing, and storage.



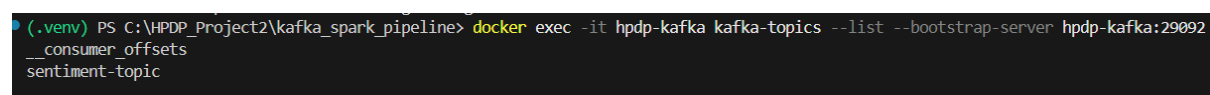
CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
03466e22bc94	docker.elastic.co/kibana/kibana:8.13.4		"/bin/tini -- /usr/l..."	About an hour ago	Up About an hour	0.0.0.0:5601->5
601/tcp, [::]:5601->5601/tcp	hdp-kibana					
b7b50bf770e2	confluentinc/cp-kafka:7.6.1		"/etc/confluent/dock..."	About an hour ago	Up About an hour	0.0.0.0:9092->9
092/tcp, [::]:9092->9092/tcp	hdp-kafka					
b26a5ebee91	confluentinc/cp-zookeeper:7.6.1		"/etc/confluent/dock..."	About an hour ago	Up About an hour	0.0.0.0:2181->2
181/tcp, [::]:2181->2181/tcp	hdp-zookeeper					
63f597a8c42e	docker.elastic.co/elasticsearch/elasticsearch:8.13.4		"/bin/tini -- /usr/l..."	About an hour ago	Up About an hour	0.0.0.0:9200->9
200/tcp, [::]:9200->9200/tcp	hdp-elasticsearch					

Figure 4.2 Docker Containers for Kafka, Zookeeper, Elasticsearch, and Kibana

4.3 Kafka Streaming Layer

Figure 4.3 shows the Kafka topic created for this project. The topic was named sentiment-topic and was used as the communication channel between the Kafka producer and Spark Structured Streaming. The cleaned review records are sent into this topic as streaming messages. Each message represents one review record and contains the review metadata together with the cleaned review text.

By using a Kafka topic, the system can separate the data ingestion process from the processing layer. This means the Kafka producer only focuses on sending messages, while Spark Structured Streaming focuses on consuming and processing them. As shown in Figure 4.3, the sentiment-topic topic was successfully created and listed in Kafka. This confirms that the streaming ingestion layer was prepared before sending review data into the pipeline.



```
(.venv) PS C:\HPDP_Project2\kafka_spark_pipeline> docker exec -it hdp-kafka kafka-topics --list --bootstrap-server hdp-kafka:29092
__consumer_offsets
sentiment-topic
```

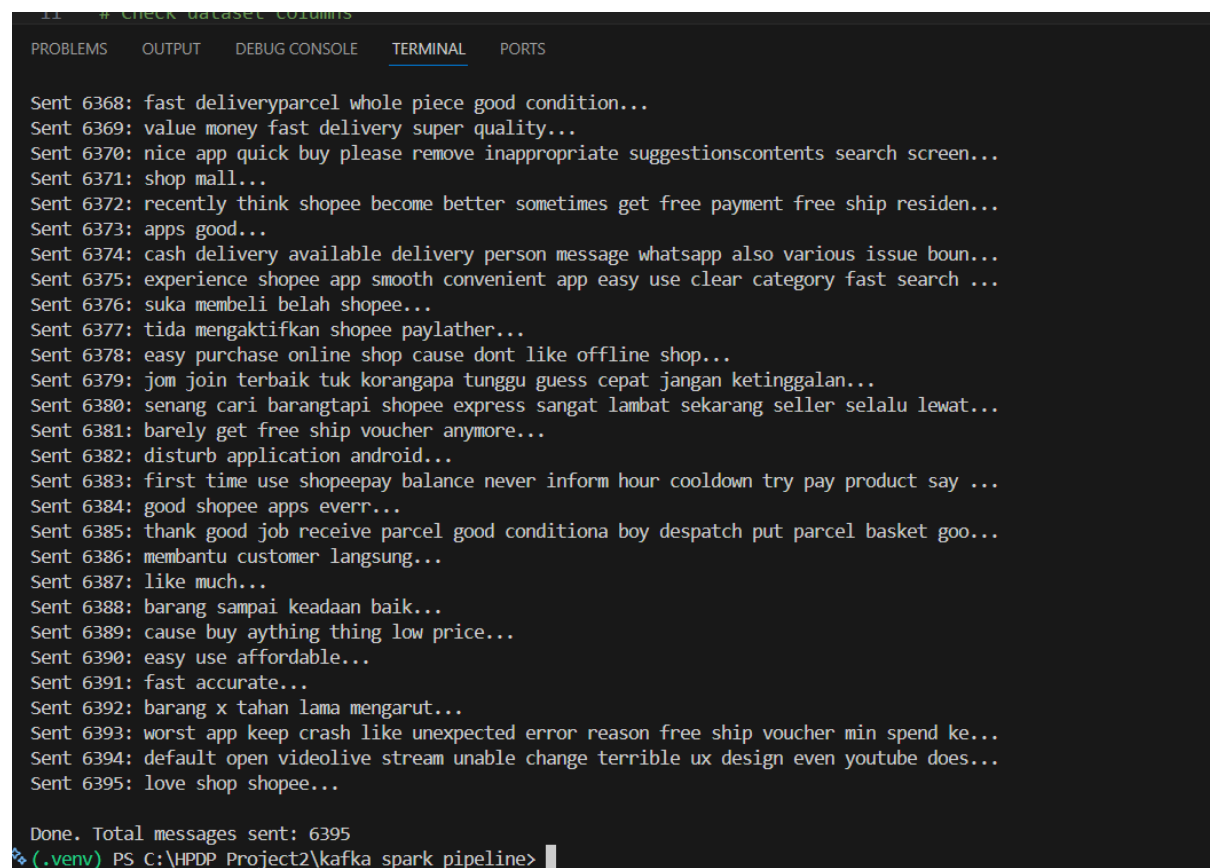
Figure 4.3 Kafka Topic Created for Streaming Review Data

4.4 Kafka Producer for Review Data Ingestion

Figure 4.4 shows the Kafka producer sending cleaned review data to the Kafka topic. The producer was implemented using Python and reads records from the cleaned dataset generated during the preprocessing stage. Each record is converted into a structured JSON message

containing attributes such as review_id, rating, review_date, app_version, thumbs_up_count, original_text, cleaned_text, and produced_timestamp.

The producer sends the records one by one to simulate real-time customer review ingestion. This is important because real customer reviews are usually generated continuously rather than all at once. As shown in Figure 4.4, the producer successfully sent 6395 cleaned review records to the sentiment-topic. This confirms that the data ingestion component was able to stream the full cleaned dataset into Kafka for further processing by Spark.



```
11 # Check dataset columns
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Sent 6368: fast deliveryparcel whole piece good condition...
Sent 6369: value money fast delivery super quality...
Sent 6370: nice app quick buy please remove inappropriate suggestionscontents search screen...
Sent 6371: shop mall...
Sent 6372: recently think shopee become better sometimes get free payment free ship residen...
Sent 6373: apps good...
Sent 6374: cash delivery available delivery person message whatsapp also various issue boun...
Sent 6375: experience shopee app smooth convenient app easy use clear category fast search ...
Sent 6376: suka membeli belah shopee...
Sent 6377: tida mengaktifkan shopee paylather...
Sent 6378: easy purchase online shop cause dont like offline shop...
Sent 6379: jom join terbaik tuk korangapa tunggu guess cepat jangan ketinggalan...
Sent 6380: senang cari barangtapi shopee express sangat lambat sekarang seller selalu lewat...
Sent 6381: barely get free ship voucher anymore...
Sent 6382: disturb application android...
Sent 6383: first time use shopeepay balance never inform hour cooldown try pay product say ...
Sent 6384: good shopee apps evrr...
Sent 6385: thank good job receive parcel good conditiona boy despatch put parcel basket goo...
Sent 6386: membantu customer langsung...
Sent 6387: like much...
Sent 6388: barang sampai keadaan baik...
Sent 6389: cause buy aything thing low price...
Sent 6390: easy use affordable...
Sent 6391: fast accurate...
Sent 6392: barang x tahan lama mengarut...
Sent 6393: worst app keep crash like unexpected error reason free ship voucher min spend ke...
Sent 6394: default open videolive stream unable change terrible ux design even youtube does...
Sent 6395: love shop shopee...

Done. Total messages sent: 6395
(.venv) PS C:\HPDP_Project2\kafka_spark_pipeline>
```

Figure 4.4 Kafka Producer Sending Cleaned Review Data to Kafka Topic

4.5 Spark Structured Streaming Processing Layer

Figure 4.5 shows the Spark Structured Streaming job processing Kafka messages. Spark continuously listens to the sentiment-topic topic and consumes the incoming review messages. After receiving the data, Spark parses the JSON message format and extracts the relevant fields required for prediction. The trained sentiment classification model is then loaded and applied to the cleaned_text field to predict the sentiment of each review.

The prediction process produces a numerical label and maps it into a readable sentiment category such as positive, negative, or neutral. In addition, processing timestamps are added to the records to indicate when the streaming data was processed. As shown in Figure 4.5, Spark successfully processed the Kafka messages in multiple micro-batches and saved the prediction

output into `streaming_prediction.csv`. This confirms that the real-time processing layer was working correctly and that the trained model could be integrated into the streaming pipeline.

```
(.venv) PS C:\HPDP_Project2\kafka_spark_pipeline> $env:PYSARK_PYTHON="C:\HPDP_Project2\.venv\Scripts\python.exe"
>> $env:PYSARK_DRIVER_PYTHON="C:\HPDP_Project2\.venv\Scripts\python.exe"
>>
>> spark-submit `
>> --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.1.1 ` ...
26/07/09 16:01:43 INFO Utils: C:\Users\yeewe\AppData\Local\Temp\spark-95b0b4d4-173f-4f58-a51c-f2301827d61c\userFiles-2570681d-8501-45bc-8318-9d9ba1508b45\fetchFileTemp7602700478659125341.tmp has been previously copied to C:\Users\yeewe\AppData\Local\Temp\spark-95b0b4d4-173f-4f58-a51c-f2301827d61c\userFiles-2570681d-8501-45bc-8318-9d9ba1508b45\com.google.code.findbugs-jsr305-3.0.0.jar
26/07/09 16:01:43 INFO Executor: Adding file:/C:/Users/yeewe/AppData/Local/Temp/spark-95b0b4d4-173f-4f58-a51c-f2301827d61c\userFiles-2570681d-8501-45bc-8318-9d9ba1508b45\com.google.code.findbugs-jsr305-3.0.0.jar to class loader default
26/07/09 16:01:43 INFO Utils: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService' on port 55240.
26/07/09 16:01:43 INFO NettyBlockTransferService: Server created on Yeewen:55240
26/07/09 16:01:43 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replication policy
26/07/09 16:01:43 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, Yeewen, 55240, None)
26/07/09 16:01:43 INFO BlockManagerMasterEndpoint: Registering block manager Yeewen:55240 with 434.4 MiB RAM, BlockManagerId(driver, Yeewen, 55240, None)
26/07/09 16:01:43 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, Yeewen, 55240, None)
26/07/09 16:01:43 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, Yeewen, 55240, None)
26/07/09 16:01:49 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
Spark streaming started.
Listening to Kafka topic: sentiment-topic
Press Ctrl + C to stop.
```

Figure 4.5 Spark Structured Streaming Processing Kafka Messages

4.6 Elasticsearch Storage Layer

Figure 4.6 shows the sentiment prediction records stored in Elasticsearch. After Spark generates the prediction results, the output records are written into the Elasticsearch index named `sentiment_predictions`. Each stored document contains review metadata, cleaned review text, predicted label, predicted sentiment, and timestamp information.

Elasticsearch is used because it supports efficient indexing, searching, and retrieval of semi-structured data. This makes it suitable for storing streaming prediction results that may later be queried or visualized in a dashboard. As shown in Figure 4.6, the prediction records were successfully indexed in Elasticsearch under the `sentiment_predictions` index. This confirms that the Spark streaming output was delivered to the storage layer and could be retrieved for Kibana validation and dashboard analysis.

```

{
  "hits" : {
    "total" : {
      "value" : 6395
    },
    "hits" : [
      {
        "_source" : {
          "review_id" : "12c34620-7baa-4041-85a9-2fc1ff43a864",
          "rating" : 5,
          "predicted_label" : 2,
          "predicted_sentiment" : "Positive",
          "processed_timestamp" : "2026-07-09T08:02:57.825191+00:00"
        }
      },
      {
        "_source" : {
          "review_id" : "f02f60c0-ee63-4526-9486-ab476587cc45",
          "rating" : 5,
          "predicted_label" : 2,
          "predicted_sentiment" : "Positive",
          "processed_timestamp" : "2026-07-09T08:02:57.825191+00:00"
        }
      },
      {
        "_source" : {
          "review_id" : "c4a59bfe-d2e9-44e9-a91b-d8efa563cb8b",
          "rating" : 3,
          "predicted_label" : 0,
          "predicted_sentiment" : "Negative",
          "processed_timestamp" : "2026-07-09T08:02:57.825191+00:00"
        }
      }
    ]
  }
}

```

Figure 4.6 Sentiment Prediction Records Stored in Elasticsearch

4.7 Summary of Apache System Architecture

Overall, the Apache system architecture successfully connects the data ingestion, streaming processing, prediction, and storage components into one complete workflow. Kafka acts as the real-time messaging layer, Spark Structured Streaming performs sentiment classification using the trained model, and Elasticsearch stores the processed prediction results. The use of Docker also simplifies service deployment and ensures that the system components can run together in a consistent environment. Therefore, the architecture provides a suitable foundation for real-time sentiment analysis, dashboard visualization, and comparison between batch and streaming processing.

5.0 Dashboard Analysis and Results

The batch and streaming prediction outputs were imported into Power BI for visualization and analysis. Power BI was selected because it supports interactive dashboards, calculated measures, filtering and could visualize clear comparison between different processing modes. Four dashboard pages were developed to separate the batch analysis, detailed prediction results, streaming analysis and batch with streaming comparison to present the findings in a clear sequence. The batch dataset was used to evaluate the full prediction result whereas the raw streaming dataset was retained to analyse the Kafka, Spark micro-batch activity.

5.1 Batch Sentiment Overview

Figure 5.1 presents the batch sentiment overview dashboard in Power BI. This dashboard shows the full batch prediction results after the selected sentiment classification model was applied to the cleaned Shopee review dataset. The dashboard includes the total number of reviews, batch prediction accuracy, correct predictions, incorrect predictions, predicted sentiment distribution and sentiment prediction confusion matrix.

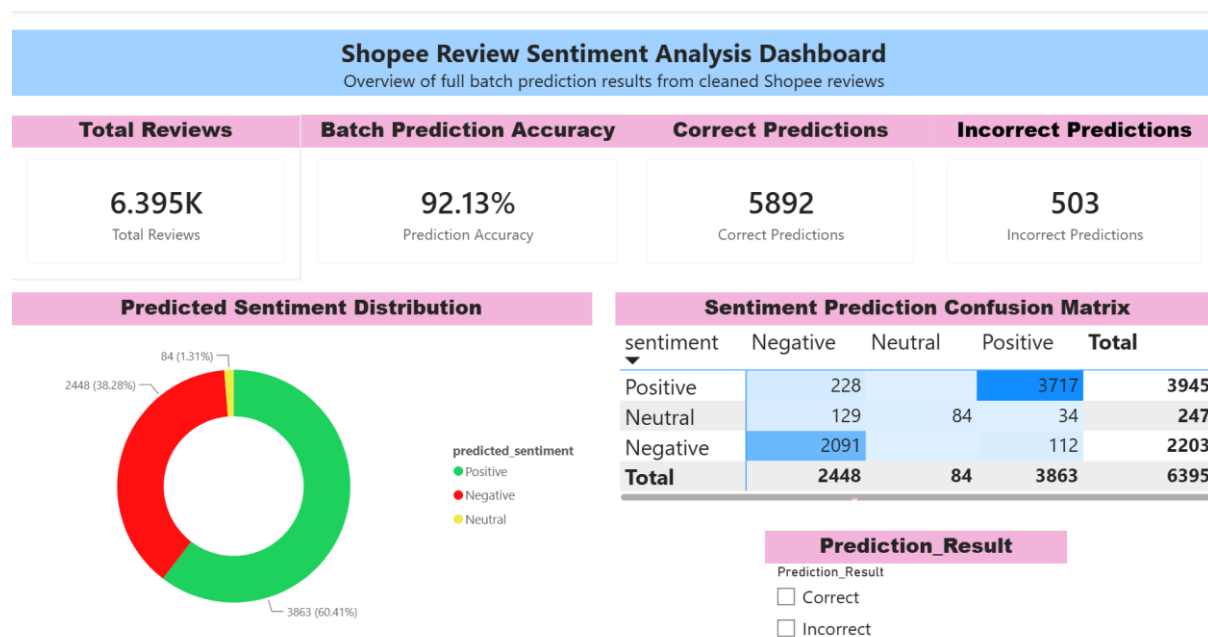


Figure 5.1 Batch Sentiment Overview Dashboard in PowerBI

A total of 6395 Shopee reviews were included in the full batch prediction output. Among these reviews, 5892 reviews were correctly predicted, while 503 reviews were incorrectly predicted. This produced a batch prediction accuracy of 92.13%. This value represents the matching result between the actual sentiment labels and predicted sentiment labels for the full batch dataset used in the dashboard. It should not be the same accuracy with the formal model testing accuracy in Section 3.0 because Section 3.0 evaluates the model using the testing subset after the train and test sets split while this dashboard summarizes the prediction result for the combined batch records.

The donut chart shows that positive sentiment was the most common predicted class with 3863 reviews of the total reviews. Then followed by negative sentiment with 2448 reviews. Meanwhile, only 84 reviews were predicted as neutral. This indicates that the overall batch prediction output was mainly dominated by positive reviews, although a significant portion of negative feedback was also detected.

The sentiment prediction confusion matrix compares the actual sentiment which was derived from customer ratings, with the predicted sentiment which was generated by the Multinomial Naive Bayes model based on review text. Based on the matrix, the model correctly predicted

3717 positive reviews, 2091 negative reviews and 84 neutral reviews. However, several misclassifications also occurred. For example, 228 actual positive reviews were predicted as negative, while 112 actual negative reviews were predicted as positive.

Overall, the batch sentiment overview dashboard provides a clear summary of the full batch sentiment prediction results. The result shows that the selected model was able to produce a high overall batch prediction match rate but the neutral sentiment remained the most difficult class to identify. This is likely because the neutral class had a much smaller number of samples compared with the positive and negative classes. Therefore, the dashboard is useful not only for showing the overall sentiment trend but also for identifying the model's weakness in classifying minority sentiment classes.

5.2 Detailed Batch Prediction Analysis

Figure 5.2 presents the detailed batch prediction analysis dashboard. This page contains the predicted sentiment distribution by rating and a review prediction details table.

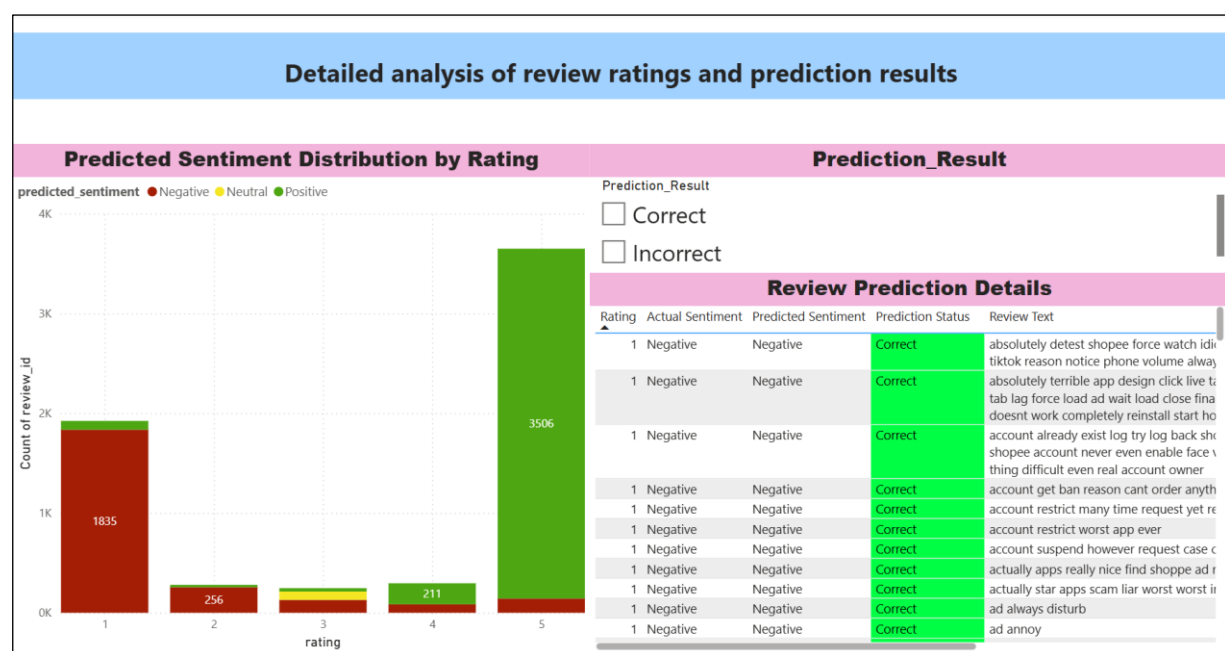


Figure 5.2 Detailed Batch Prediction Analysis Dashboard in Power BI

The rating analysis showed that most one-star and two-star reviews were predicted as negative reviews while most four-star and five-star reviews were predicted as positive reviews. This reflects the expected relationship between customer rating and sentiment. However, some ratings contained more than one predicted sentiment category because the Multinomial Naive Bayes model analysed the review text instead of relying directly on the numerical rating.

For example, a five-star review may contain positive overall feedback but may also mention problems such as payment failure, application errors or delivery delays. As a result, the written

text may be predicted as negative review even though the rating is high. The review details table allows users to inspect the actual sentiment, predicted sentiment, rating and prediction result for individual reviews.

An interactive slicer to filter the correct and incorrect predictive result was also included so that users can display only correct or incorrect predictions. This makes it easier to identify model errors and examine the review text that caused misclassification.

5.3 Streaming Sentiment Overview

5.3.1 Kibana Streaming Output Validation

The streaming prediction output generated by Apache Spark was stored in Elasticsearch and visualized using Kibana. Kibana was used as a validation and monitoring tool to confirm that the streaming records were successfully indexed and could be searched and analysed. The exported `streaming_prediction.csv` file was imported into a local Elasticsearch index named `sentiment_predictions_local`, and a Kibana data view was created to access the prediction fields.

The Kibana dashboard included three main visualizations: streaming sentiment distribution, predicted sentiment by review rating and messages processed by timestamp. These visualizations were selected to validate the sentiment output and demonstrate how the streaming records were processed over time.

5.3.1.1 Streaming Sentiment Distribution

Figure 5.3 presents the streaming sentiment distribution in Kibana. A total of 60 streaming samples were visualized. There are 42 positive reviews and 18 negative reviews. No neutral sentiment was present in the streaming output. This may be caused by the small sample size and the limited representation of neutral reviews.

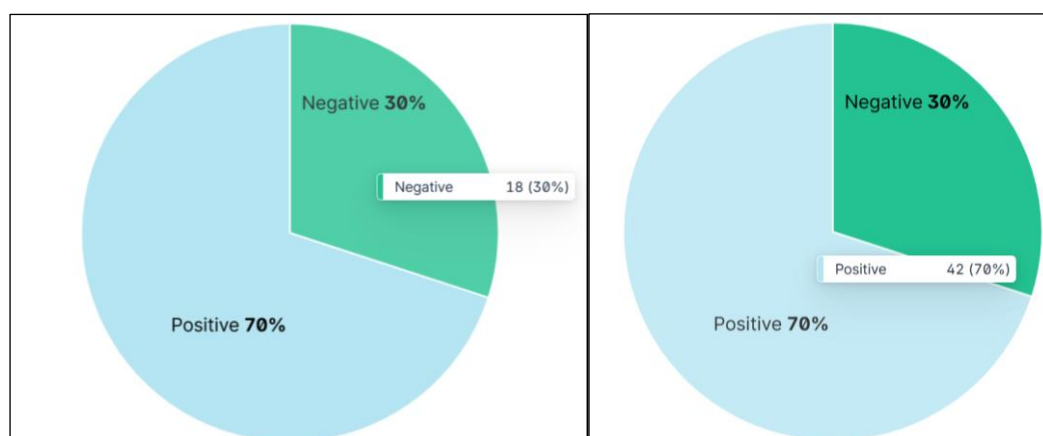


Figure 5.3 Streaming Sentiment Distribution in Kibana

The result indicates that positive sentiment was dominant in the streaming sample. However, this distribution was based on 20 unique reviews repeated across 60 streaming reviews.

Therefore, the result represents the streamed reviews distribution rather than the distribution of a large independent review sample.

5.3.1.2 Predicted Sentiment by Review Rating

Figure 5.4 shows the predicted sentiment according to review rating. Only ratings 1, 3 and 5 appeared in the streaming sample. Rating 5 contained the highest number of records and was mainly classified as positive, while ratings 1 and 3 were classified as negative. There is no neutral in this streaming samples as the model takes only the positive and negative reviews. The neutral reviews only consist of 1.31% of the full dataset, it is too small to let the model successfully take the neutral review for sampling.

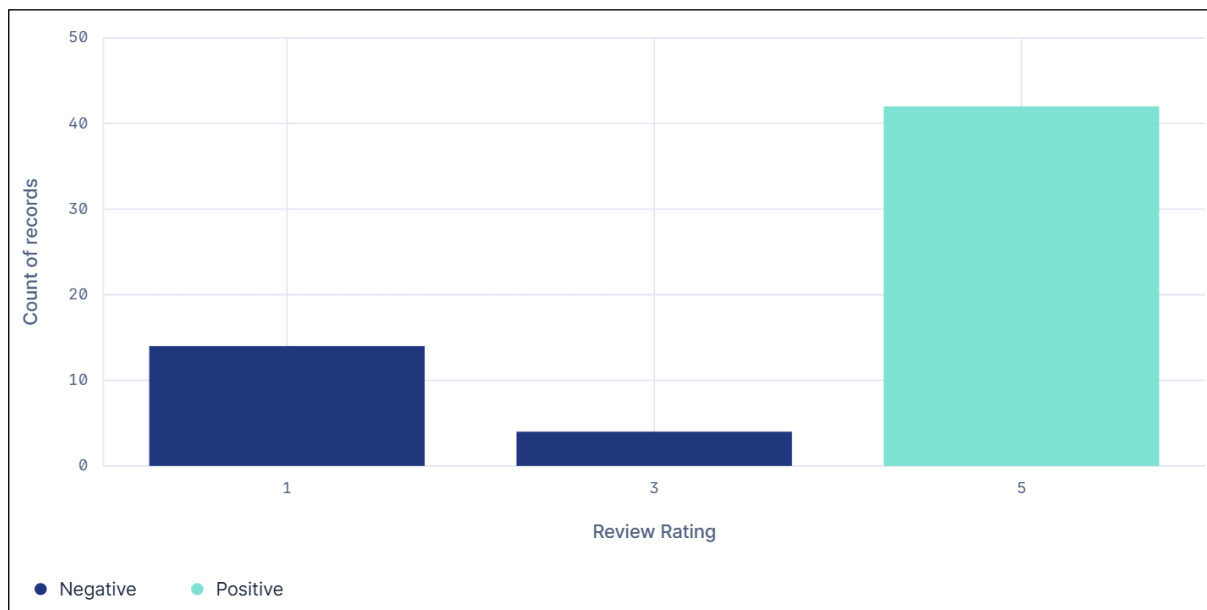


Figure 5.4. Predicted Sentiment by Review Rating in Kibana

This pattern is consistent with the expected relationship between rating and sentiment. High ratings generally represent customer satisfaction, whereas low ratings usually indicate dissatisfaction. Ratings 2 and 4 were not shown because they were not present in the selected streaming sample.

5.3.1.3 Messages Processed by Timestamp

Figure 5.5 presents the number of streaming sample reviews processed according to the recorded processing timestamp. The chart shows that the messages were processed at several different times rather than as one continuous group.

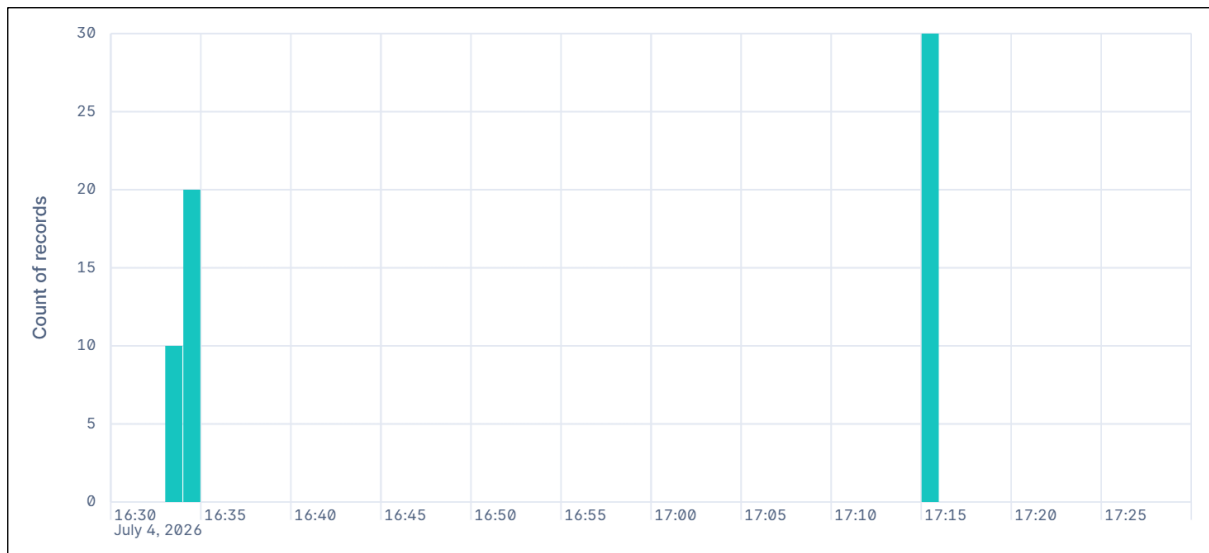


Figure 5.5. Sample Reviews Processed by Timestamp in Kibana

The larger bars indicate that more review messages were completed within the processing interval. This behaviour is consistent with Apache Spark Structured Streaming which processes incoming Kafka records in micro-batches. The timestamp visualization therefore provides evidence that the streaming messages were successfully received, processed and stored in Elasticsearch.

Kibana was mainly used to validate the Elasticsearch storage and streaming output. Power BI was then used to provide a more detailed analytical overview including unique-review accuracy and comparison with batch processing.

5.3.2 Power BI Streaming Sentiment Overview

Figure 5.6 presents the streaming sentiment analysis overview dashboard in Power BI. This dashboard was developed to summarize the sentiment prediction results generated from the Kafka and Spark Structured Streaming pipeline. It includes the total number of streaming messages, the number of unique streaming reviews, streaming accuracy, predicted sentiment distribution and the number of messages processed per Spark micro-batch.

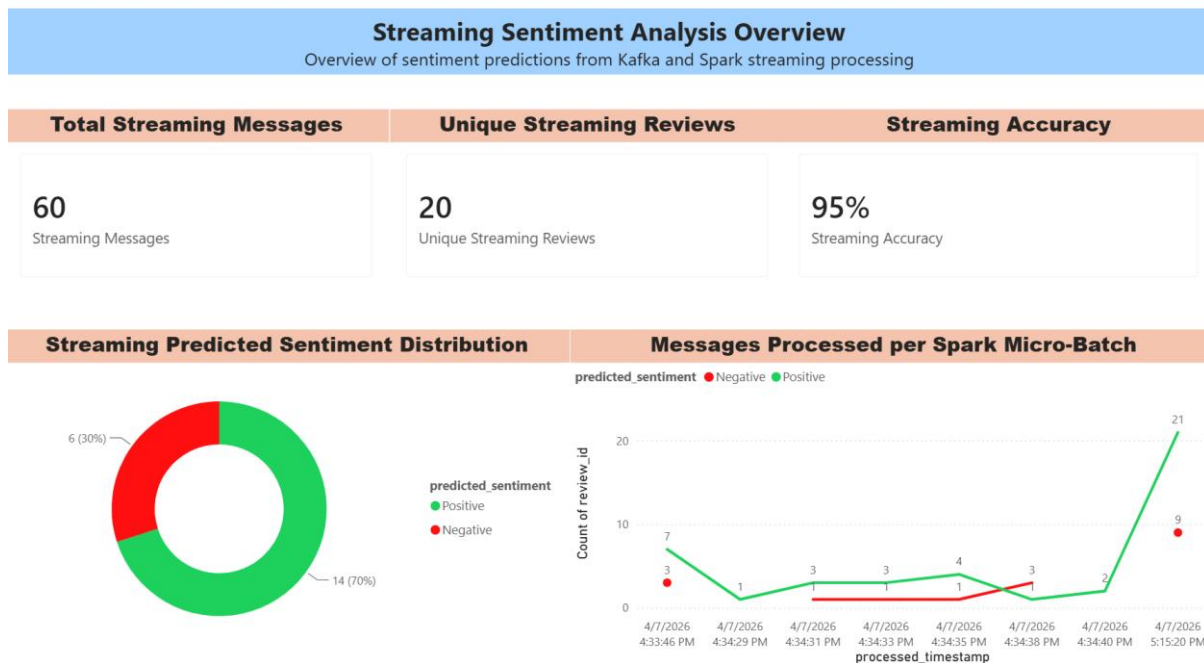


Figure 5.6 Streaming Sentiment Analysis Dashboard in PowerBI

A total of 60 streaming messages were processed by the streaming pipeline. However, these messages represented 20 unique Shopee reviews. The streaming accuracy was 95%, which indicates that most of the unique streaming reviews were correctly classified by the sentiment model. However, this accuracy should be interpreted carefully because the streaming dataset was much smaller than the full batch dataset. The Kafka producer has limit it into 20 review messages only.

The streaming predicted sentiment distribution shows that positive sentiment was the dominant class. Out of the 20 unique streaming reviews, 14 reviews were predicted as positive, while 6 reviews were predicted as negative. No neutral sentiment was predicted in the streaming output. This may be due to the small streaming sample size and the limited number of neutral reviews in the original dataset.

The line chart shows the number of messages processed by Spark micro-batch over time. The number of processed messages varied at different timestamps, showing that Spark Structured Streaming handled the incoming Kafka messages in micro-batches instead of processing all records at once. This confirms that the streaming pipeline was able to continuously receive, process and classify review messages from Kafka.

Overall, the Power BI streaming dashboard shows that the real-time sentiment analysis pipeline was successfully implemented. Kafka was able to stream the review messages, Spark Structured Streaming was able to classify the incoming text and the prediction results were successfully visualized in Power BI. The result also shows that positive sentiment remained the main predicted category in the streaming sample but a larger and more balanced streaming dataset would be needed to provide a more reliable streaming sentiment distribution.

5.4 Batch and Streaming Comparison

Figure 5.7 presents the comparison between batch and streaming sentiment analysis in Power BI. The dashboard compares the batch prediction results with the streaming prediction results to show the differences between both processing modes. This comparison includes prediction accuracy and sentiment distribution across batch and streaming outputs.

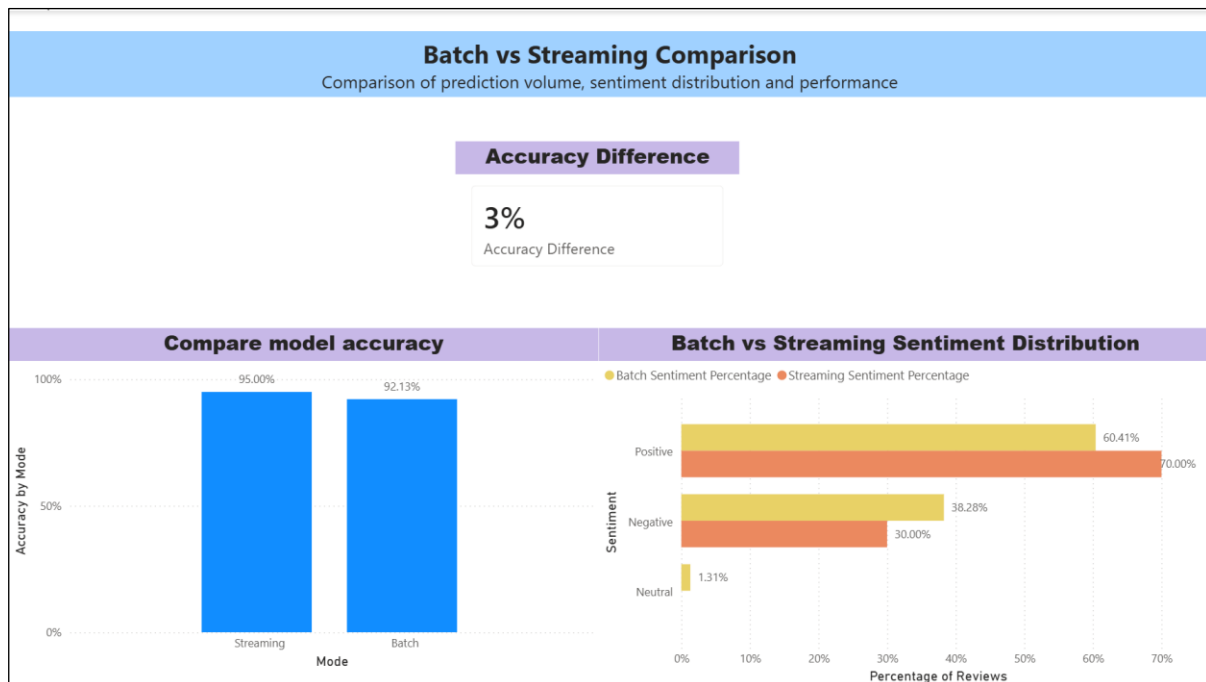


Figure 5.7 Batch and streaming sentiment analysis comparison in PowerBI

For batch processing, the selected sentiment model was applied to the full cleaned batch dataset which contained 6395 Shopee reviews. The batch prediction accuracy was 92.13%. In contrast, the streaming pipeline processed 60 Kafka messages with 20 unique Shopee reviews repeated during the streaming test. Based on these unique streaming reviews, the streaming prediction accuracy was 95.00%.

Although the streaming accuracy was slightly higher than the batch prediction accuracy, this result should be interpreted carefully. The batch result was calculated using a much larger dataset while the streaming result was based on a smaller sample of 20 unique reviews. Therefore, the higher streaming accuracy does not necessarily mean that streaming processing produced a better model performance. Instead, it mainly shows that the Kafka and Spark Structured Streaming pipeline was able to receive, process, classify and store incoming review messages successfully.

The sentiment distribution also showed a similar overall pattern between both processing modes. In batch processing, 60.41% of the reviews were predicted as positive, 38.28% were predicted as negative and 1.31% were predicted as neutral. In streaming processing, 70.00% of the unique reviews were predicted as positive and 30.00% were predicted as negative while no neutral sentiment was predicted. This may be due to the smaller streaming sample and the limited representation of neutral reviews in the streamed data.

Overall, both batch and streaming processing showed that positive sentiment was the dominant category, followed by negative sentiment. However, the comparison should be viewed as a system-level comparison rather than a strict model performance comparison because the batch and streaming outputs were based on different data volumes. The batch dashboard provides a broader view of the full dataset while the streaming dashboard demonstrates that the real-time sentiment analysis pipeline can classify incoming review data through Kafka, Spark Structured Streaming and Elasticsearch.

5.5 Summary

Overall, the dashboard analysis presents the sentiment prediction results generated from both batch processing and streaming processing. In the batch analysis, a total of 6,395 cleaned Shopee reviews were processed using the selected sentiment classification model. The batch dashboard showed a batch prediction accuracy of 92.13% with 5892 correct predictions and 503 incorrect predictions. However, this accuracy value represents the full batch prediction result used for visualization not the model testing accuracy reported in Section 3.0.

The batch sentiment distribution showed that most reviews were predicted as positive, followed by negative while neutral sentiment formed the smallest group. This result is consistent with the original dataset distribution where positive reviews were the majority and neutral reviews were the minority because of the small number of neutral samples in the dataset.

For streaming analysis, the Kafka and Spark Structured Streaming pipeline successfully processed incoming review messages and stored the prediction results in Elasticsearch. The streaming dashboard showed that positive sentiment was also the dominant category in the streamed data. However, the streaming result was based on a smaller sample compared with the full batch dataset.

In summary, the dashboards successfully visualize sentiment results and compare batch and streaming outputs. The batch dashboard provides a wider overview of sentiment patterns across the full cleaned dataset while the streaming dashboard demonstrates that the system can classify incoming reviews in near real time using Kafka, Spark Structured Streaming and Elasticsearch with similar output with batch mode. The results also highlight that future improvement should focus on increasing neutral sentiment samples and improving the model's ability to classify minority sentiment classes.

6.0 Optimization and Comparison

This section explains the optimization and performance comparison of the sentiment analysis system. The comparison focuses on batch processing and streaming processing based on processing time, throughput, classification accuracy, CPU utilization and memory consumption. These metrics were used to evaluate the efficiency and behaviour of the system under different processing modes.

6.1 Batch and Streaming Comparison

Table 6.1 shows the performance comparison between batch mode and streaming mode. Batch processing was used to classify the cleaned Shopee review dataset in one complete process while streaming processing classified incoming review records through Kafka and Spark Structured Streaming.

Table 6.1 Batch and Streaming Performance Comparison

Performance Metric	Batch Processing	Streaming Processing
Total records processed	6395 records	6395 records
Total processing time	0.3968 seconds	9.0977 seconds
System throughput	16,116.71 records/second	702.92 records/second
CPU utilization rate	95.90%	6.01%
Peak memory consumption	236.23 MB	240.29 MB
Classification accuracy	88.19%	95.00% based on 20 streaming sample
Elasticsearch success documents	Not applicable	406 documents
Elasticsearch failed documents	Not applicable	0 documents

Based on Table 6.1, batch processing achieved a much shorter processing time of 0.3968 seconds compared with streaming processing which required 9.0977 seconds. This is because batch processing processes the dataset directly in one execution while streaming processing involves additional components such as Kafka message ingestion, Spark micro-batch processing and Elasticsearch storage. Therefore, streaming processing requires more time due to the extra communication and storage steps in the pipeline.

In terms of throughput, batch processing achieved 16,116.71 records per second, while streaming processing achieved 702.92 records per second. This shows that batch processing was faster for processing a fixed dataset because all records were processed together. In contrast, streaming processing handled records in micro-batches which is slower but more suitable for continuous real-time data ingestion.

The CPU utilization result shows that batch processing used a higher CPU utilization rate of 95.90% while streaming processing showed a CPU utilization rate of 6.01% only. This indicates that batch processing used more CPU intensively in a short period of time while streaming processing distributed the workload across micro-batches over a longer period. The peak memory consumption for both modes was quite similar, with 236.23 MB for batch processing and 240.29 MB for streaming processing.

For classification accuracy, batch processing recorded a model classification accuracy of 88.19%. This value refers to the formal model evaluation accuracy from the testing dataset. Meanwhile, the streaming dashboard showed a streaming accuracy of 95.00%, it was calculated

using a smaller streaming sample. Therefore, the streaming accuracy mainly shows that the streaming pipeline was able to classify incoming records correctly during the test rather than proving that streaming mode is more accurate than batch mode.

Overall, batch processing was more efficient in terms of processing time and throughput, making it suitable for analysing a complete historical dataset. Streaming processing was slower due to micro-batch processing and system integration overhead but it successfully supported real-time sentiment classification. The Elasticsearch result also showed 406 successful documents and 0 failed documents, confirming that the streaming output was successfully stored in Elasticsearch.

6.2 Model Optimization and Improvement

The main weakness of the selected Multinomial Naive Bayes model was its poor performance in identifying neutral sentiment. Based on Table 3.1 and Table 3.2 from Section 3.0, although the model achieved the highest overall accuracy and weighted F1-score, the neutral class recorded very low performance because neutral reviews represented only a very small proportion of the dataset. As a result, many actual neutral reviews were classified as either negative or positive.

Table 3.1 Performance Comparison of Sentiment Classification Models

Model	Accuracy	Precision	Recall	F1-Score	Macro Average F1-score
Multinomial Naive Bayes	88.19%	0.8538	0.8819	0.8663	0.59
Logistic Regression	84.75%	0.8629	0.8475	0.8475	0.63
Linear SVM	86.40%	0.8574	0.8640	0.8592	0.64

Table 3.2 Pre-class performance by for Multinomial Naive Bayes

Class	Precision	Recall	F1-Score	Support
Negative	0.79	0.91	0.85	441
Neutral	0.00	0.00	0.00	49
Positive	0.94	0.92	0.93	789

To improve this limitation, future work should collect more neutral reviews or apply resampling techniques to create a more balanced dataset. Oversampling the neutral class, undersampling the majority classes or applying suitable class-balancing methods could help the model learn neutral sentiment more effectively. In addition, advanced models such as LSTM, BiLSTM, BERT or DistilBERT can be tested because these models are better at understanding sentence context and may improve the classification of neutral or mixed sentiment reviews.

The text representation could also be improved. The current model uses TF-IDF which is efficient and suitable for traditional machine learning but it has limitations in understanding context, mixed opinions, sarcasm and word relationships. For example, a review may contain both positive and negative phrases and TF-IDF may not fully capture the overall meaning. Future work could explore word embeddings or transformer-based models such as BERT, DistilBERT or multilingual BERT. These models may perform better for mixed Malay-English reviews because they can understand contextual meaning more effectively.

6.3 Architecture Optimization and Improvement

The current architecture successfully integrated Kafka, Spark Structured Streaming, Elasticsearch, Kibana and Power BI. However, the streaming test used only 20 unique reviews repeated across 60 messages. Future testing should use a larger number of independent reviews to provide a more realistic evaluation of the streaming pipeline.

In addition, the Kafka and Spark Structured Streaming pipeline was successfully implemented and was able to classify review messages in real time. However, the streaming mode required a longer processing time compared with batch processing because the data passed through several components including Kafka, Spark and Elasticsearch.

To optimize the streaming architecture, the Spark micro-batch interval can be tuned to reduce processing delay. Kafka producer settings can also be improved by adjusting batch size and message delay to control the flow of incoming records. Furthermore, Spark checkpointing can be used to improve fault tolerance and allow the streaming job to recover more safely if an error occurs.

The Elasticsearch storage layer can also be optimized by using proper index mapping and bulk insertion settings. This would help improve write performance when more streaming records are processed. Monitoring tools can also be added to track CPU usage, memory usage, throughput and processing latency more clearly during future experiments.

6.4 Overall Discussion

Overall, the comparison shows that batch processing and streaming processing have different strengths. Batch processing achieved faster processing time and higher throughput because it processed the full dataset directly in one execution. Therefore, it is more suitable for historical analysis and full dataset reporting.

Streaming processing required more time because it processed data through Kafka, Spark Structured Streaming and Elasticsearch. However, streaming processing is more suitable for real-time use case because it can continuously receive and classify new customer reviews as they arrive. The successful Elasticsearch output also confirms that the streaming pipeline was able to store prediction results correctly.

In conclusion, the batch mode is more efficient for analysing large completed datasets while the streaming mode is more useful for real-time sentiment monitoring. Future improvements should focus on improving neutral sentiment classification, increasing the streaming sample size, tuning Spark micro-batch settings and adding more detailed system monitoring for CPU, memory and processing latency.

7.0 Conclusion & Future Work

7.1 Conclusion

The project successfully for Shopee Malays by integrating natural language processing (NLP), machine learning, Apache Kafka, Apache Spark structured stream, Elasticsearch, Kibana and Power BI Ia Google Play Review has developed a real-time emotional analysis system. A total of 10,000 customer comments were collected, pre-processed, and converted into a clean data set containing 6,395 valid comments for emotional classification.

Under the same experimental conditions, three machine learning models were evaluated, using Multinomial Naive Bayes, Logical Regression and Linear Support Vector Machine. Among them, Multinomial Naive Bayes achieves the best overall performance with the highest weighted F1 score and powerful computing efficiency, making it the most suitable model for deployment in real-time streaming media pipelines.

The proposed Apache-based architecture has successfully demonstrated continuous data ingestion, real-time mood prediction and storage of prediction results through Kafka, Spark structured stream and Elasticsearch. The prediction results are further visualized using Kibana and Power BI dashboards, allowing users to monitor customer emotions from the perspective of batch and streaming. The comparison of batch and streaming shows that although the accuracy of streaming is slightly higher, the difference is mainly affected by the smaller size of the streaming sample. Nevertheless, both processing models have produced a consistent emotional trend, and positive comments represent most of the customer feedback.

Overall, the project has successfully achieved all the proposed goals and shows that the combination of machine learning with Apache streaming technology provides an effective solution for real-time emotional analysis. The developed system provides a practical basis for organizations to continuously monitor customer opinions and support data-driven decision-making.

7.2 Improvement

Some improvements can be considered to further enhance the sentiment classification model and real-time processing pipeline.

First, future studies should collect larger and more balanced data sets, especially by increasing the number of neutral reviews. A more balanced data set will improve the model's ability to classify minority emotional categories and produce more reliable assessment results.

Secondly, more advanced natural language processing technology can be explored. Although TF-IDF works effectively on traditional machine learning models, converter-based models such as BERT, DistilBERT or multilingual BERT can capture contextual information, irony and mixed language expressions more effectively, which may lead to higher classification accuracy.

From the perspective of system architecture, many real-time review data can be used instead of duplicate samples to evaluate streaming media pipelines. Kafka partitioning, Spark structured flow parameters and resource allocation can also be optimized to increase throughput, reduce processing delay and enhance system scalability.

In addition, the monitoring ability of the system can be enhanced by implementing automatic error handling, checkpoint recovery, performance monitoring and real-time dashboard refresh. These enhancements will improve the reliability and maintainability of streaming pipelines in the production environment.

Finally, the proposed framework can be extended to analyse customer reviews from multiple e-commerce platforms or social media sources, to achieve a more comprehensive customer emotional monitoring system to support business intelligence and decision-making across different online platforms.

References

Jain, P. K., & Pamula, R. (2020, August 24). *A systematic literature review on machine learning applications for consumer sentiment analysis using online reviews*.

arXiv.org. <https://arxiv.org/abs/2008.10282v1>

Poornima, S., Shashidhar, K., & Fernandes, R. (2025). Real Time Sentiment Analysis of

YouTube Live Chat With Apache Spark and Kafka. *IEEE Access*, 13, 188929–

188938. <https://doi.org/10.1109/access.2025.3625785>

Appendix

Appendix A: Elasticsearch Server Running on Localhost

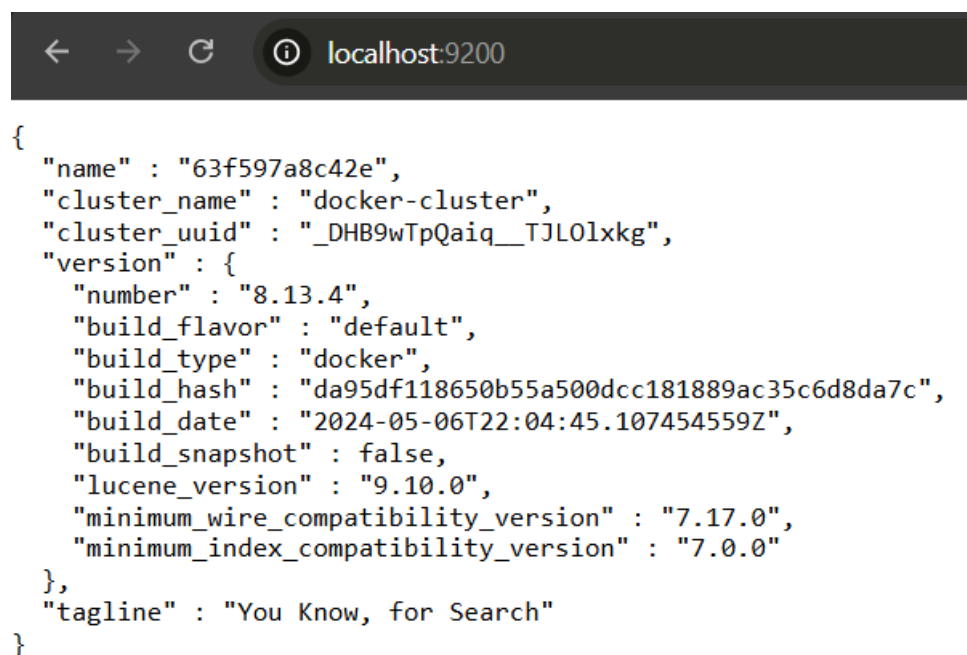


Figure A.1 Elasticsearch Server Running on Localhost

Presentation Youtube Link: https://youtu.be/vumdt10_g0A

